DSA LAB 3

Data Structures and Algorithms

Graded Lab Tasks: Doubly Linked Lists

Complete C++ solutions with original questions, explanatory comments, sample outputs, and time/space complexity notes.

Lab Task 1 • Reverse a Doubly Linked List

Lab Task 2 • Swap Two Nodes Without Swapping Data

Lab Task 3 • Convert Singly to Doubly Linked List

Lab Task 1: Reverse a Doubly Linked List

Question

Write a function which reverses the order of a doubly linked list.

Approach: Reverse the direction of each node's next and prev pointers. Then update the head pointer to the former last node. This reverses the actual list without swapping node data.

C++ Solution

```
#include <iostream>
using namespace std;

// Define a node of a doubly linked list.
struct Node {
    int data;
    Node* prev;
    Node* next;

    Node(int value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

// Insert a new node at the end of the list.
void insertAtEnd(Node*& head, int value) {
    Node* newNode = new Node(value);

    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* current = head;

    while (current->next != nullptr) {
        current = current->next;
    }

    current->next = newNode;
    newNode->prev = current;
}

// Reverse the doubly linked list by changing its pointers.
void reverseList(Node*& head) {
    Node* current = head;
    Node* temp = nullptr;

    while (current != nullptr) {
        // Exchange the next and previous pointers.
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        // Move to the next node in the original list.
        current = current->prev;
    }

    // Update head to the former last node.
    if (temp != nullptr) {
        head = temp->prev;
    }
}

// Display the list from beginning to end.
void displayList(Node* head) {
    Node* current = head;

    while (current != nullptr) {
        cout << current->data << " ";
        current = current->next;
    }

    cout << endl;
}

int main() {
    Node* head = nullptr;
    int n, value;

    cout << "Enter the number of nodes: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Enter value: ";
        cin >> value;
        insertAtEnd(head, value);
    }

    cout << "Original list: ";
```

```
    displayList(head);

    reverseList(head);

    cout << "Reversed list: ";
    displayList(head);

    return 0;
}
```

Sample Output

```
Enter the number of nodes: 4
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Original list: 10 20 30 40
Reversed list: 40 30 20 10
```

Complexity: O(n) time and O(1) extra space.

Lab Task 2: Swap Two Nodes in a Doubly Linked List

Question

Write a function which takes two values as input from the user and searches them in the list. If both values are found, swap both the nodes in which these values are found. You are not supposed to swap values.

Approach: Find the first node containing each value, then change the nodes' links. The data fields remain unchanged. This solution handles adjacent nodes, non-adjacent nodes, and nodes at the beginning or end of the list.

C++ Solution

```
#include <iostream>
using namespace std;

// Define a node of a doubly linked list.
struct Node {
    int data;
    Node* prev;
    Node* next;

    Node(int value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

// Insert a node at the end of the list.
void insertAtEnd(Node*& head, int value) {
    Node* newNode = new Node(value);

    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* current = head;

    while (current->next != nullptr) {
        current = current->next;
    }

    current->next = newNode;
    newNode->prev = current;
}

// Find the first node containing the given value.
Node* searchNode(Node* head, int value) {
    Node* current = head;

    while (current != nullptr) {
        if (current->data == value) {
            return current;
        }

        current = current->next;
    }

    return nullptr;
}

// Swap two nodes without exchanging their data.
void swapNodes(Node*& head, Node* first, Node* second) {
    if (first == nullptr || second == nullptr || first == second) {
        return;
    }

    // Ensure that first comes before second when they are adjacent.
    if (second->next == first) {
        swapNodes(head, second, first);
        return;
    }

    // Handle adjacent nodes separately.
    if (first->next == second) {
        Node* before = first->prev;
        Node* after = second->next;

        if (before != nullptr) {
            before->next = second;
        } else {
            head = second;
        }

        second->prev = before;
        second->next = first;
        first->prev = second;
        first->next = after;

        if (after != nullptr) {
            after->prev = first;
        }
```

```
        return;
    }

    // Save the neighboring nodes.
    Node* firstPrev = first->prev;
    Node* firstNext = first->next;
    Node* secondPrev = second->prev;
    Node* secondNext = second->next;

    // Connect the neighbors of the first node to the second.
    if (firstPrev != nullptr) {
        firstPrev->next = second;
    } else {
        head = second;
    }

    if (firstNext != nullptr) {
        firstNext->prev = second;
    }

    // Connect the neighbors of the second node to the first.
    if (secondPrev != nullptr) {
        secondPrev->next = first;
    } else {
        head = first;
    }

    if (secondNext != nullptr) {
        secondNext->prev = first;
    }

    // Exchange the nodes' positions.
    first->prev = secondPrev;
    first->next = secondNext;
    second->prev = firstPrev;
    second->next = firstNext;
}

// Display the linked list.
void displayList(Node* head) {
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }

    cout << endl;
}

int main() {
    Node* head = nullptr;
    int n, value, firstValue, secondValue;

    cout << "Enter the number of nodes: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Enter value: ";
        cin >> value;
        insertAtEnd(head, value);
    }

    cout << "Original list: ";
    displayList(head);

    cout << "Enter the first value to swap: ";
    cin >> firstValue;

    cout << "Enter the second value to swap: ";
    cin >> secondValue;

    Node* first = searchNode(head, firstValue);
    Node* second = searchNode(head, secondValue);

    if (first == nullptr || second == nullptr) {
        cout << "One or both values were not found." << endl;
    } else {
        swapNodes(head, first, second);

        cout << "List after swapping nodes: ";
        displayList(head);
    }

    return 0;
}
```

Sample Output

```
Enter the number of nodes: 5
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value: 50
Original list: 10 20 30 40 50
Enter the first value to swap: 20
Enter the second value to swap: 50
List after swapping nodes: 10 50 30 40 20
```

Complexity: O(n) time for searching and O(1) extra space for swapping.

Lab Task 3: Convert a Singly Linked List into a Doubly Linked List

Question

Write a function that takes a singly linked list as a parameter and creates a doubly linked list containing the same data present in the singly linked list. In other words, the singly linked list will be converted into a doubly linked list.

Approach: Traverse the singly linked list and create a new doubly linked node for every existing node. Connect each new node using both prev and next pointers. The original singly linked list remains unchanged.

C++ Solution

```
#include <iostream>
using namespace std;

// Define a node for the singly linked list.
struct SNode {
    int data;
    SNode* next;

    SNode(int value) {
        data = value;
        next = nullptr;
    }
};

// Define a node for the doubly linked list.
struct DNode {
    int data;
    DNode* prev;
    DNode* next;

    DNode(int value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

// Insert a node at the end of the singly linked list.
void insertSingly(SNode*& head, int value) {
    SNode* newNode = new SNode(value);

    if (head == nullptr) {
        head = newNode;
        return;
    }

    SNode* current = head;

    while (current->next != nullptr) {
        current = current->next;
    }

    current->next = newNode;
}

// Convert a singly linked list into a new doubly linked list.
DNode* convertToDoubly(SNode* head) {
    DNode* doublyHead = nullptr;
    DNode* doublyTail = nullptr;

    SNode* current = head;

    while (current != nullptr) {
        // Create a doubly linked node with the same data.
        DNode* newNode = new DNode(current->data);

        if (doublyHead == nullptr) {
            // Initialize the first node.
            doublyHead = newNode;
            doublyTail = newNode;
        } else {
            // Connect the new node to the previous node.
            doublyTail->next = newNode;
            newNode->prev = doublyTail;
            doublyTail = newNode;
        }

        // Move to the next node in the original list.
        current = current->next;
    }

    return doublyHead;
}

// Display the singly linked list.
void displaySingly(SNode* head) {
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }

    cout << endl;
```

```
}

// Display the doubly linked list.
void displayDoubly(DNode* head) {
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }

    cout << endl;
}

int main() {
    SNode* singlyHead = nullptr;
    int n, value;

    cout << "Enter the number of nodes: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Enter value: ";
        cin >> value;
        insertSingly(singlyHead, value);
    }

    cout << "Original singly linked list: ";
    displaySingly(singlyHead);

    // Create the new doubly linked list.
    DNode* doublyHead = convertToDoubly(singlyHead);

    cout << "Converted doubly linked list: ";
    displayDoubly(doublyHead);

    return 0;
}
```

Sample Output

```
Enter the number of nodes: 4
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Original singly linked list: 10 20 30 40
Converted doubly linked list: 10 20 30 40
```

Complexity: O(n) time and O(n) extra space for the new doubly linked list.

Key Concepts for the Lab Viva

Concept	Explanation
head	Points to the first node of the list.
next	Points to the next node.
prev	Points to the previous node in a doubly linked list.
Node*& head	Passes the head pointer by reference so a function can update it.
Node*	A pointer to a node.
nullptr	Represents a pointer that points to no object.
Dynamic memory	The new operator creates nodes at runtime.

Important: Each code listing is a separate, complete C++ program. Save and compile each one separately because every program has its own main() function.